



Ciri: Scaling-aware job scheduling for small language models on over-committed GPU clusters

Jingxin Wang[✉], Shuyuan Fan, Zhao Zhang^{✉*}

Rutgers University, 94 Brett Rd, Piscataway, 08854, NJ, USA

ARTICLE INFO

Keywords:

Scheduling
Small language models
GPU clusters

ABSTRACT

Despite the remarkable breakthroughs in large language models (LLMs), researchers are also designing models with modest size (hundreds of millions to billions of parameters) for domain science and agentic AI. Training and fine-tuning modest models on over-committed GPU clusters can experience long turnarounds due to the high computational demand and extensive queue wait times. Enabled by gradient accumulation, such a workload can leverage any number of GPUs as long as the global mini-batch size is fixed. Training with more GPUs can reduce execution time, but leads to longer queue wait times and higher node hour consumption due to nonlinear scaling efficiency. There is a clear tradeoff between turnaround time and the node hour consumption, which is agnostic to existing schedulers. In this paper, we present Ciri, a reinforcement learning-enabled and scaling-aware user-level job scheduler, which incorporates workload scaling curves and machine states into scheduling decisions. Ciri dynamically selects the GPU count to trade off turnaround time for node hour consumption. We evaluate Ciri with three GPT models: GPT-345M, GPT-1.5B, and GPT-2.0B with job traces of the TACC Lonestar6 and Frontera GPU clusters. The results show that Ciri can effectively reduce the node hour consumption by 0.9–1.8% for every 1% increase in turnaround across the two machines.

1. Introduction

Our age has witnessed the success of large language models (LLMs), such as conversational AI [1], mathematical reasoning [2], and code generation [3]. Researchers are also designing models with modest scales, i.e., with hundreds of millions to a few billions of parameters [4,5], for science and engineering problems. Small language models (SLMs) also play a critical role in Agentic AI [6]. In the university setting, scientists and engineers heavily rely on the federally invested graphics processing unit (GPU) clusters for model training, fine-tuning, and inference. Training SLMs consumes a substantial amount of node hours. Each GPU cluster is equipped with a few hundred GPUs, and these GPU clusters are almost always overloaded, as illustrated by Fig. 2. For a scientist, the excessively long queue wait time and the inherently high computational demand of neural networks collectively lead to prolonged experiment turnarounds.

On the other hand, SLMs can be trained with any number of GPUs as long as the global mini-batch size remains constant, which is enabled by the gradient accumulation technique. However, these training workloads exhibit a non-linear scaling trend due to the inter-GPU communication. Fig. 1 shows the strong scaling efficiency of GPT-345M, GPT-1.5B, and GPT-2.0B [7] across A100 GPUs. The efficiency of GPT-2.0B on 128 A100s is as low as 37%.

The unique combination of long queue wait time and limited scaling efficiency presents an opportunity to trade off turnarounds (TAT) for lower node hour consumptions (NHC). It is feasible to train an SLM at a smaller scale, which yields a higher scaling efficiency and lower NHC. However, the increase in TAT can be modest, given the already long queue wait time. This tradeoff is critical for domain scientists and engineers, who commonly face a shortage of computational budgets on the national cyberinfrastructure of ACCESS [8] or the NAIRR Pilot [9]. The efficient tradeoff between TAT and NHC also enhances overall machine utilization if widely adopted by users.

The state of practice is to train an SLM with a fixed scale, which results in either long TAT or high NHC. Another strawman solution may predict the queuing delay, as in classic works [10–12], and optimize job submission accordingly. In this way, it is possible to optimize a single objective, e.g., either TAT or NHC. However, it is challenging to design an efficient tradeoff strategy between the two objectives simultaneously. Queue time prediction itself is unreliable due to the stochastic nature of job arrivals, queuing dynamics, and execution durations (see Section 4). Empirical evidence [13] shows that prediction accuracy ranges from 20% to 72%, with over 91% of estimates deviating from actual queue delays. This unpredictability makes it difficult to support

* Corresponding author.

E-mail addresses: jw1755@scarletmail.rutgers.edu (J. Wang), sf850@scarletmail.rutgers.edu (S. Fan), zhao.zhang@rutgers.edu (Z. Zhang).

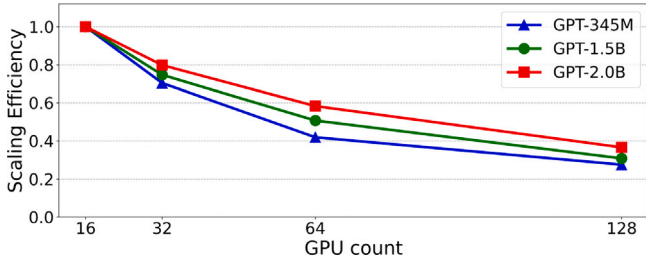


Fig. 1. Scaling efficiency of GPT-345M, GPT-1.5B and GPT-2.0B on NVIDIA A100 GPUs.

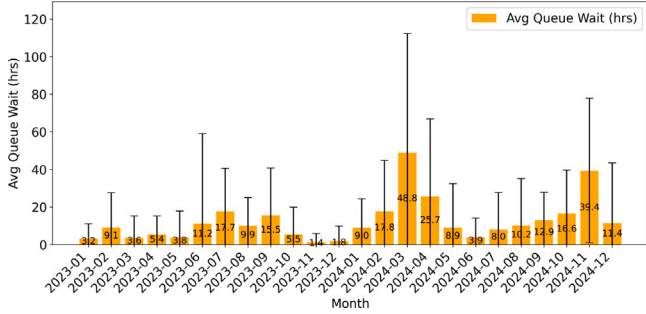


Fig. 2. Monthly average queue wait time with standard deviation on TACC Lonestar6.

consistent decision-making, especially when optimizing both TAT and NHC under dynamic cluster loads. Although existing works [14–18] can optimize the overall timespan or node hour consumption as a single optimization objective, the decisions overlook users' requirements and tolerance for possible increases in TAT.

In this work, we focus on the user's perspective and formulate the user-level job scheduling problem as a Markov Decision Process (MDP) by viewing the scale (i.e., the number of GPUs) selection as a decision action. Two unique data sources enable this decision action: (1) the present and past job traces on production GPU clusters and (2) the scaling curve of the applications. We present Ciri, a user-level scheduling framework that learns to adapt scales based on job-specific scaling behavior and historical system states. Ciri combines a set of Transformer-based [19] queue wait time predictors with a learning agent trained using the actor-critic method. Ciri exploits a configurable reward function as a weighted sum of the two optimization objectives, enabling an efficient tradeoff between TAT and NHC.

We evaluate Ciri using three GPT models with varying sizes: GPT-345M, GPT-1.5B, and GPT-2.0B on the TACC Lonestar6 cluster with the job trace of 01/2023-12/2024. For single-objective optimization, Ciri achieves a 10.6% lower average TAT across the entire validation range compared to the baseline. When the machine is heavily loaded, the reduction in TAT is as high as 16.1% (see Section 7.2). For dual optimization objectives of TAT and NHC, Ciri can efficiently trade off longer TAT for significantly lower NHC. For example, Ciri reduces the overall NHC of GPT-2.0B by 8.9%, 21.8%, and 54.9% with a TAT increase of 5.4%, 22.3%, and 50.7%, respectively, across various weight combinations between the two objectives. For GPT-1.5B, Ciri reduces the NHC by 10.9%, 24.1%, 59.1% with TAT increase of 6.9%, 13.1%, and 35.2%, respectively. For every 1% increase in TAT, Ciri can reduce NHC by 1.7% (see Section 7.3.2). For the generalization of Ciri effectiveness, we perform a similar study on the TACC Frontera GPU subsystem with the job trace of 12/2019-08/2021. The tradeoff ratio between NHC and TAT is 1.5 and 1.7 for GPT-2.0B and GPT-1.5B, respectively. (see Section 7.3.3)

We would like to note that Ciri is a user-level scheduler, not a system-wide scheduler. Individual users can leverage Ciri to reduce the node hour consumption for the SLM workload. The measured effectiveness scope of Ciri is limited to the Lonestar6 GPU cluster, as the models are trained and validated with the job traces of Lonestar6. However, the methods of Ciri are generally effective across clusters that are over-committed. It requires a base model and policy training with the particular job trace of a target machine. The applicability of Ciri is beyond SLM training. Scientists can also use this tool for long-running simulations or data analysis workflows, achieving lower node hour consumption with a measured scaling profile. The contributions of this study include:

- a Transformer-based queue wait time predictor,
- a mixture-of-experts architecture as the base model for policy gradient (PG) and actor-critic (AC),
- a scaling-aware scheduler that leverages historical job traces to efficiently trade off TAT for NHC savings,
- a flexible system design to enable various combinations of base models and RL methods,
- and the evaluation of Ciri design using real-world applications and a production machine.

2. Background

In this section, we discuss the challenges of job scheduling and the opportunities presented by gradient accumulation and job traces. We also introduce the candidate RL method to balance the turnaround time and node hour consumption.

National cyberinfrastructure efforts, such as ACCESS and NAIRR Pilot, have funded several GPU clusters with mode sizes (i.e., hundreds of GPUs). These efforts are to meet the computational needs of AI-driven scientific and engineering research, which has shown promising results from astronomy to zoology. Noticable examples include AlphaFold2 [20], GenSLMs [21], and MetNet-3 [22]. Unlike the extreme LLMs, these models have hundreds of millions to a few billion parameters. Training these models on federally funded GPU clusters is time-consuming (weeks to months). For example, it takes 128 A100 GPUs for 11 days to train OpenFold [4], which is an open-source AlphaFold2 implementation. To comply with existing scheduling policies on clusters, such as the maximum wall clock time of 48 h, it is common practice for users to break long-running workloads into shorter jobs. Despite the feasibility of the aforementioned approach, scientists are inevitably encountering the following obstacles. Despite the success of the aforementioned research, scientists are inevitably encountering the following obstacles.

Long wait times for large experiments. Computing centers like TACC are experiencing long queue wait times, a trend that is expected to continue, as shown in Fig. 2. For example, the average queue wait time was over 40 h in 03/2024 on TACC Lonestar6. The prolonged queue delays disrupt workload continuity, degrading responsiveness and the overall quality of service (QoS).

Tradeoff between Turnaround Time(TAT) and node hour consumption (NHC). SLM and LLM training can usually run with multiple scales thanks to the gradient accumulation technique. However, waiting for different numbers of compute nodes has significantly varying queue wait times, and the corresponding execution times differ across scales with a non-linear pattern. These two factors result in vastly different experiment turnaround times (including waiting and running), which is commonly agnostic to domain scientists. Deep learning (DL) practitioners may lose the opportunity to consume fewer node hours at the cost of a modest increase in turnaround time.

2.1. Opportunities

We observe two key opportunities to address the limitations of current solutions. First, gradient accumulation enables deep learning models to train at arbitrary job scales (i.e., number of GPUs), allowing the flexibility to scale jobs across different numbers of nodes. Training the same model with various scales exhibits non-linear scaling efficiency in the sense that training with a larger scale tends to have shorter execution time but higher node hour consumption due to low scaling efficiency, and vice versa. This scaling pattern introduces the opportunity to leverage small scales for lower node hour consumption, albeit at the cost of a longer turnaround time. The second opportunity is to learn from past experience, e.g., the job trace of the Lonestar6 cluster. Many GPU clusters are over-committed, resulting in excessively long queue wait times. Waiting for fewer GPUs can be significantly shorter than waiting for more GPUs. If the scheduler can adjust the scale based on machine status, it may be able to leverage this opportunity to lower the overall node hour consumption with only a modest increase in turnaround time.

These insights motivate a new scheduling approach that moves beyond the fixed-scale strategy. We propose an RL-based scheduler that is aware of each application's scaling profile and learns to dynamically select job scales. Our solution leverages offline-trained queue predictors and online feedback to co-optimally balance turnaround time and node hour consumption on an over-committed GPU cluster.

2.2. Reinforcement learning (RL)

Reinforcement learning enables agents to learn from experience rather than explicit supervision. By iteratively taking actions, observing outcomes, and receiving feedback in the form of rewards, agents refine their policies to maximize cumulative reward. In this work, we evaluate three RL techniques: Deep Q-Network (DQN), Policy Gradient (PG), and Actor-Critic (AC).

2.2.1. Deep Q-Network

Deep Q-Learning (DQN) is a value-based reinforcement learning method [23]. It learns a function $Q_\theta(s, a)$ that gives the expected total reward if the agent takes actions a in state s , and follows the best strategy.

In our scheduler, we use a neural network to estimate Q-value for each possible GPU scale. The network takes the job and cluster state as input and predicts a value for each action. During scheduling, the agent selects the action that has the highest predicted Q-value:

$$a = \arg \max_a Q_\theta(s, a) \quad (1)$$

The learning goal is to minimize the gap between the predicted Q-values and the target values. The loss function is defined as:

$$L(\theta) = \left(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \right)^2 \quad (2)$$

Here, r represents the reward, s' denotes the next state, and γ is the discount factor. We use a target network Q_{θ^-} that is updated slowly to make training more stable. The model learns from past experiences that are stored in a replay buffer. DQN is sensitive to frequently changing environments, so we normalized the rewards for training stability.

2.2.2. Policy Gradient

The Policy Gradient method is a powerful alternative to value-based reinforcement learning approaches [23], which can learn a parameterized policy π_θ that directly maps state to action probabilities.

In a scheduler implementation, the agent's policy is represented by a neural network that takes job and cluster state as input and outputs a probability distribution over node scales. The objective is to maximize the expected cumulative reward $J(\theta)$, defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [r(\tau)], \quad (3)$$

where τ is a trajectory $(s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ sampled from the current policy π_θ , and $r(\tau) = \sum_{t=1}^T r_t$ is the cumulative reward.

The gradient of $J(\theta)$ can be expressed using the policy gradient theorem [24]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [\nabla_\theta \log \pi_\theta(\tau) \cdot r(\tau)]. \quad (4)$$

This expectation is approximated through Monte Carlo rollouts:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \cdot r^i, \quad (5)$$

where N is the number of sampled episodes and $r^i = \sum_{t=1}^{T_i} r_t^i$ is the total return for episode i .

The parameters are updated via gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta), \quad (6)$$

where α is the learning rate. Unlike value-based methods, policy gradient supports stochastic exploration and avoids the need to approximate a value function. However, they are known for high variance in gradient estimates and may reach local maxima [25,26]. We mitigate these challenges by incorporating a reward baseline and entropy regularization during training, which has been shown to stabilize learning in noisy environments.

2.2.3. Actor-Critic

The Actor-Critic method is a popular extension of policy gradient that reduces gradient variance by combining a learnable value function with a policy network [27]. The actor chooses an action based on the current state s , and the critic tells how good that state is by giving it a value $V_w(s)$. This value helps the actor determine which actions are more effective.

In our scheduler, the actor network receives job and cluster information as input and outputs a probability distribution over different node scales. At the same time, the critic network estimates the quality of the current state. The actor does not use total reward; it uses an advantage between what really happened and what the critic expected. The expectation is approximated:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \cdot A_t^i, \quad (7)$$

where the advantage is computed as:

$$A_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t).$$

To help the critic learn, we train it by minimizing the difference between its predicted value and the actual return:

$$\mathcal{L}_{\text{critic}} = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} (V_w(s_t^i) - r_t^i - \gamma V_w(s_{t+1}^i))^2. \quad (8)$$

This setup lets the scheduler learn more efficiently by combining actions from the actor and feedback from the critic. We also incorporate entropy regularization and reward normalization during training to help the agent explore more effectively and learn more stably, especially in noisy or dynamic environments.

Compared to the policy gradient method, the Actor-Critic method usually learns faster and makes more efficient scheduling decisions when the workload is complex or dynamic.

3. Related work

Existing job scheduling approaches in clusters either predict queue delay from historical traces or rely on reinforcement learning to optimize system-wide objectives such as throughput and fairness. However, these methods often assume static job graphs, fixed job scales, or centralized control, limiting their applicability to dynamic, user-driven workflows. In contrast, our work addresses a finer-grained scheduling

challenge, which involves balancing turnaround time and node hour consumption for long-running deep learning workloads under dynamic resource availability. We design a scaling-aware reinforcement learning scheduler that adapts scale decisions based on queue states, job characteristics, and system feedback, providing a flexible and general solution for the tradeoff between TAT and NHC.

3.1. State of the practice

To manage resource contention and ensure fairness, computing centers typically enforce fixed wall clock limits. For example, both TACC Lonestar6 and NERSC Perlmutter enforce a 48-hour limit. However, many SLM and LLM workloads exceed these limits and must checkpoint and resume via job arrays submitted to batch schedulers, such as SLURM [28]. The common practice is to partition the long deep learning (DL) workload into smaller jobs and then run each job with a fixed scale. This reactive approach results in either excessively long queue wait times or unnecessarily high node hour costs. For those with a tight computing budget, this approach offers limited flexibility for users to balance turnaround times and node hours.

Another line of work focuses on queue wait time prediction [10–12]. These methods train models on historical traces to forecast queue wait time and submit the following jobs accordingly. Empirical studies reveal substantial limitations for these works. Reported prediction accuracies range widely between 20% and 72%, with over 91% of predictions deviating from actual queue delays [13]. This is largely due to the inherent randomness in job arrivals, preemptions, and runtime variances. As shown in Figs. 2, 3, and 4, queue states exhibit high temporal variance, underscoring the challenge of accurately predicting queuing delays.

For DL workloads, existing research improves training efficiency and fairness in heterogeneous environments from the overall machine perspective. Heet [29] introduces a runtime-aware elastic training framework that adapts to changing system throughput, but it is tightly coupled with Horovod-based training and assumes centralized control over job scale. Similarly, the Fairness-Aware Scheduler [30] targets resource fairness by prioritizing underserved jobs in heterogeneous GPU clusters.

Taken together, existing solutions either rely on static heuristics or attempt to predict inherently unpredictable queue behaviors. They focus on system-level scheduling problems, while our goal is for users to balance between turnarounds and node hour consumption. This motivates our solution: an RL-based, scaling-aware job scheduler that dynamically adapts to cluster load, scaling behavior, and job characteristics to balance between turnaround time and node hour consumption.

3.2. Reinforcement learning in scheduling

Prior efforts in reinforcement learning in scheduling have focused on improving queue latency, optimizing throughput under network contention, or reasoning under execution uncertainty.

DRAS [14] presents a reinforcement learning scheduler that learns both backfilling and reservation strategies to reduce queuing delays while maintaining job fairness. It employs a learning-based method that does not rely on a predefined model of system dynamics. However, the approach is built for traditional HPC jobs with fixed resource requirements and does not support workloads with flexible training scales or variable job scales. A two-phase optimization strategy [15] is proposed, where tasks are first assigned to servers and then the job execution order is determined through search. This approach is effective for jobs with task dependencies but assumes fully known execution graphs and deterministic runtimes, limiting its use in training scenarios where job behavior evolves over time. ContSched [16] applies reinforcement learning to learn scheduling decisions that account for network congestion in multi-GPU systems. The method enhances throughput by minimizing communication overhead and mitigating

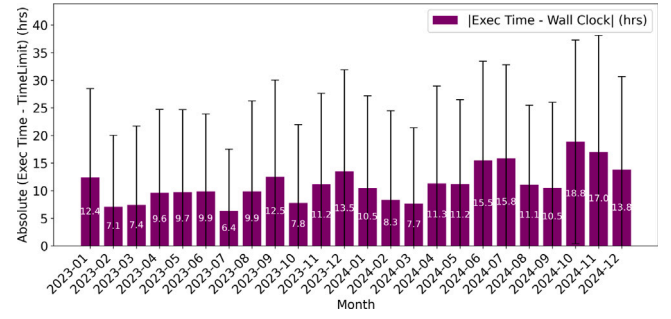


Fig. 3. Monthly deviation between execution time and time limit on TACC Lonestar6.

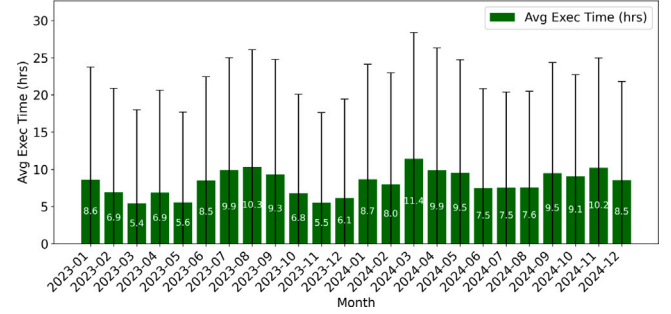


Fig. 4. Monthly average execution time with standard deviation on TACC Lonestar6.

straggler effects. Wheatley [17] focuses on decision-making under runtime uncertainty in job shop scheduling. Their method demonstrates robustness to fluctuating execution durations but is specifically tailored to manufacturing-style settings. Reinforcement learning has also been studied in the context of HPC scheduling [18], proposing a proactive scheduler that learns to submit follow-up jobs in advance by forecasting system congestion and delay. It significantly improves turnaround time for long training workloads by reducing wait time between jobs. HierRL [31] proposes a multi-level actor-critic framework for decomposing the scheduling task into cluster- and node-level actions, thereby improving scalability and sample efficiency. Attentive RL [32] integrates attention mechanisms with RL to dynamically adjust node allocation in elastic training jobs. However, both works assume workloads with fixed job graphs and do not explicitly handle turnaround time or node hour cost. None of the existing methods considers the job scaling trend and the corresponding NHC and TAT across scales. So, they are agnostic to the NHC and TAT tradeoff.

4. Job trace analysis

We develop Ciri with two job traces collected on TACC Lonestar6 and Frontera. The Lonestar6 (LS6) trace from 01/2023 to 12/2024 contains 109,234 jobs. The Frontera trace, from 12/2019 to 08/2021, has 175,093 jobs. Each job record includes requested nodes, submission time, start time, and end time. This section characterizes the statistical behaviors of these jobs, summarizes the sources of randomness, and motivates the design of our learning-based scheduler. To ease the discussion, we use LS6 as the example throughout this section. Frontera has a similar pattern.

Fig. 2 shows the high variance in queue wait time across months. In 03/2024, the average queue wait time exceeded 40 h, whereas 11/2023 and 12/2023 showed minimal queue wait times. Fig. 3 highlights deviations between a job's execution time and its requested wall clock limit. The deviations can be caused by overestimated execution time, unexpected job termination, or other factors. The standard deviation

is significant, so there is no clear pattern for the deviation between the requested and execution time. Fig. 4 illustrates a stable execution duration across months, between six and ten hours for most months. However, the high variance within each month makes the execution time hard to predict.

Overall, these results reflect the stochastic nature of real-world workloads. Job arrivals, queuing delays, and execution durations are influenced by a mixture of user actions and evolving system dynamics. In addition, manual actions, such as priority escalation, preemption, and cancellation, can further complicate the schedules. We find that predicting queue wait time using historical traces is inherently difficult. Our experiments with a set of candidate predictors show a consistently moderate error (see Section 7.1). This motivates the study of reinforcement learning, despite the potential for inaccurate predictions, to learn scheduling policies directly from feedback.

5. System design

We design a scaling-aware job scheduling framework, Ciri, for GPU clusters, which combines predictive modeling and reinforcement learning to balance turnaround time (TAT) and node hour consumption (NHC). Fig. 5 shows the architecture of Ciri, which consists of four major components: ① an offline data generator, ② a queue wait time predictor, ③ a reinforcement learning agent with multiple policy branches, and ④ a SLURM simulator [18]. These modules work together to enable both single-objective and dual-objective optimization.

5.1. Operating modes

Ciri operates in two modes: training and inference. In the training mode, Ciri first generates offline data to train the queue wait time predictors at discrete scales, such as 4, 8, 16, 32 nodes. Second, Ciri trains the candidate predictors of Random Forest, XGBoost, and the Transformer model using the offline data. Third, the Ciri agent abstracts a SLM training workload with a predefined node hour budget, e.g., 16 nodes for 4.58 days for GPT-1.5B. Fourth, the Ciri agent integrates the queue wait time predictors with policies, such as the greedy policy, deep Q-network, policy gradient, and actor-critic. Ciri then trains the policies by interacting with the SLURM simulator. The Ciri agent makes decisions on job scales (i.e., number of nodes) based on the predicted queue wait time and the node hour consumption across scales. Then Ciri submits the job to the SLURM simulator. Upon the job completion, Ciri records the TAT and NHC, subtracts NHC from the overall budget, and then selects the scale for the next job. This is an iterative process, and an episode concludes when the node hour budget is zero. The accumulated TAT and NHC will be transformed into rewards, and the agent will adjust its variables accordingly. In inference mode, Ciri takes the machine status from the past 48 h and the application node hour budget as inputs, then makes job scale decisions.

5.2. Offline data generator

As shown in Fig. 5(①), the offline data generator is responsible for producing queue wait time training samples. The job trace on Lonestar6 spans from 01/2023 to 12/2024. We use 01/2023-06/2024 as the training partition and 07/2024-12/2024 as the validation partition to examine Ciri's effectiveness. The training data for queue wait time predictors are generated as follows: (1) The SLURM scheduler loads the training job traces. (2) We sample a timestamp uniformly from the training time range. (3) The SLURM scheduler is set to start 48 h prior to the sampled timestamp to reestablish the machine state. (4) We submit a job with a selected scale at the sampled timestamp, and record the queue wait time of the job as the label. (5) The machine states are recorded as a time series of features. In particular, the machine state is defined as a 36-element vector. Each vector encodes queue status, cluster utilization, and job-level features derived from a system

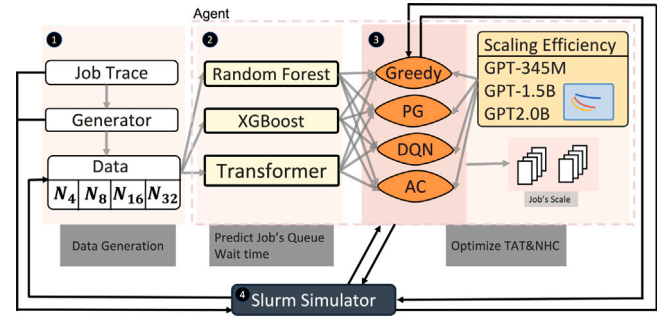


Fig. 5. Ciri Architecture overview. Ciri consists of the offline data generator (①), the reinforcement learning agent, which has the queue wait time predictor (②) and the Policy heads (③), and a SLURM simulator (④).

snapshot: The **queue state** includes the number of pending jobs, the total number of nodes requested, and summary statistics (0th, 25th, 50th, 75th, and 100th percentiles) for the time limits of pending jobs, the wait times in the queue, and the expected runtime. The **running job state** includes the number of active jobs, total occupied nodes, and the corresponding summary statistics for running jobs' time limits, wait times, and elapsed runtime. The **target job features** include the job time limit and the requested number of nodes. For each timestamp, we record 288 state vectors at 10-minute intervals to construct the machine states over a 48-hour period. The 48-hour time window is determined based on the operational policy of the supercomputer center (TACC Lonestar6), where the maximum wall-clock time allowed for a single job is 48 h. (6) We store the training data separately for the job scales of 4, 8, 16, and 32 nodes. The validation data is generated similarly.

5.3. Queue wait time predictor

This module corresponds to Fig. 5(②). It is responsible for predicting the queue wait time of a submitted job under different scale configurations. We then train three queue wait time predictors using Random Forest, XGBoost, and the Transformer model, with the generated training and validation data with distinct job scales. The results are reported in Section 7.1.

5.4. Transformer architecture

The Transformer model, also highlighted in Fig. 5(②), is used as the predictor for queue wait time. Fig. 6 shows the model details and multiple branches for job scheduling strategies. We adopt a Transformer-based neural network to estimate queue wait times. Transformers are well-suited for modeling sequential dependencies, making them ideal for learning temporal patterns in job scheduling environments.

At each decision point t , the model receives a state matrix $S_t \in \mathbb{R}^{k \times m}$, where $k = 288$ corresponds to 48 h of historical snapshots sampled at 10-minute intervals, and $m = 36$ denotes the dimensionality of each snapshot. Each row of S_t captures summary statistics over pending and running jobs, including resource demand, runtime limits, and cluster load, as well as features of the submitted job.

The input matrix is first embedded into a higher-dimensional representation via a linear projection layer and then processed through a stack of Transformer encoder layers. Each encoder layer comprises multi-head self-attention, feedforward layers, and normalization. The attention mechanism enables the model to identify salient patterns in system history (e.g., bursts of job arrivals, periodic behavior).

The output of the encoder stack is flattened and passed to four policy branches. The greedy branch produces a single action based on the greedy strategy (see Section 6.2 for details). The deep Q-network branch uses a two-layer multi-layer perceptron (MLP) to estimate the Q-value for all actions. Then it selects the action with the highest Q-value.

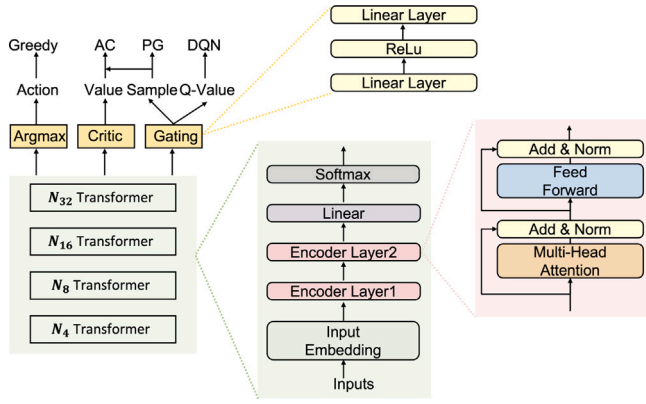


Fig. 6. The transformer base model architecture.

The policy-gradient branch applies a softmax classifier to generate an action probability distribution over possible scheduling decisions. The Actor-critic branch consists of two components: an *actor* and a *critic*. Although the actor and critic use the same encoded input, they pass through two-layer MLPs with ReLU activations separately. The actor produces a softmax distribution for all actions and samples one action for scheduling. The critic predicts the value of the current state as a baseline to stabilize training.

The greedy branch does not require additional online training with job traces. For policy gradient, we perform online training using the Adam optimizer with a fixed learning rate of 10^{-5} and the L1 loss. The AC branch is trained with the actor and critic loss. The actor learns to take actions that maximize the reward, while the critic learns to minimize the mean squared error (MSE) between its predicted value and the actual return. The training data is the same as queue wait time prediction, as discussed in Section 5.3. Model hyperparameters, such as the number of encoder layers, attention heads, embedding dimensions, and feedforward size, are selected empirically and kept consistent across all experiments.

5.5. Policy heads

Fig. 5(⊙) illustrates the policy heads. The Policy Gradient scheduler operates in a 4-way discrete action space, corresponding to the selection of one of four job scales: {4, 8, 16, 32} nodes. At each decision point, the policy selects a job scale for the incoming job.

We implement a stochastic policy using the policy gradient (PG) algorithm. The policy network is a mixture-of-experts architecture, where each expert is a Transformer-based predictor for a specific node scale. The gating module dynamically routes the input state to each expert and aggregates their outputs to produce a unified policy representation. The final layer outputs a softmax distribution over the action space.

The input state is a matrix $S_t \in \mathbb{R}^{288 \times 36}$ that encodes historical cluster and queue statistics over a 48-hour window. At runtime, the PG network receives S_t and samples a scheduling action from the predicted distribution. This stochastic decision process enables the policy to explore multiple scaling configurations, improving robustness under uncertainty. Unlike deterministic policies (e.g., DQN), which always select the highest-valued action, the stochastic policy allows randomized scale selection. This is especially useful in high-variance environments where queue wait times are difficult to predict precisely. Our policy serving module continuously samples actions at each scheduling step, adapting to dynamic workloads in an online manner.

5.6. Actor-Critic and action space

The Actor-Critic branch in Fig. 5(⊙) extends the policy gradient with a learned value baseline. The Actor-Critic scheduler also operates in a 4-way discrete action space, just like the Policy Gradient. The actor network employs a stochastic policy implemented as a mixture of experts (MoE) model, which is identical to the Policy Gradient approach. The critic uses the same MoE structure as the actor. Instead of action probabilities, it outputs a scalar value $V(s)$ that estimates the expected total reward from the current state. A simple linear layer is added on top to produce this value. The critic works as a baseline to help the actor learn more stably and mitigate randomness in the training process.

Both actor and critic take the same input state: a matrix $S_t \in \mathbb{R}^{288 \times 36}$ encoding 48 h of historical job trace and cluster information. At each step, the actor samples an action based on the softmax output, and the critic provides feedback by predicting state value. Since the actor and critic share the same structure, they can learn from the same information and improve together.

To improve stability and exploration, we add entropy regularization in the loss and normalize the discounted rewards before training. We train the Actor-Critic model using gradient descent, updating both the actor and critic after each episode (see Section 2.2.3).

5.7. Reward function

The reinforcement learning agent is trained using two reward formulations, depending on whether the optimization is with single objective or dual objectives.

In the single objective setting, the reward function targets job latency, defined as the turnaround time (TAT). At each step, the agent receives a scalar reward:

$$R = -(\text{QueueWaitTime} + \text{ExecutionTime}) \quad (9)$$

This setting encourages the policy to reduce queuing delay and execution latency jointly.

In the dual-objective setting, the scheduler also considers resource efficiency in terms of node hour consumption. The reward function is extended to include both turnaround time and resource cost:

$$R = -((1 - \alpha) \cdot T\hat{A}T + \alpha \cdot N\hat{H}C) \quad (10)$$

where $\alpha \in [0, 1]$ balances between TAT and NHC. We refer to the NHC weight as α . Here, $T\hat{A}T$ and $N\hat{H}C$ denote the normalized turnaround time and node hour consumption, respectively. Normalization is performed using exponential moving averages of the mean and standard deviation, updated across episodes. This enables the agent to adjust its scale selection strategy in response to non-uniform magnitudes of TAT and NHC.

During the warmup phase, reward normalization statistics are estimated from buffered samples. After initialization, statistics are updated online at the completion time of each job. This dynamic reward enables the policy to balance between TAT and NHC without requiring external labels or additional system changes.

5.8. Optimization modes

Ciri supports two optimization modes: single-objective and dual-objective. Each mode defines a distinct reward formulation and evaluation metric, while sharing the same action and state representation.

Single objective optimization. In this mode, the scheduler aims to minimize job turnaround time, which is the sum of queue wait time and execution time. The reinforcement learning agent is trained using the reward function defined in Eq. (9). The three base models {RF, XGB, Trans} for queue wait time predictors are combined with the three learning methods {DQN, PG, AC}. The baselines of RF, XGB, and Trans are constructed by integrating with the greedy policy to select a

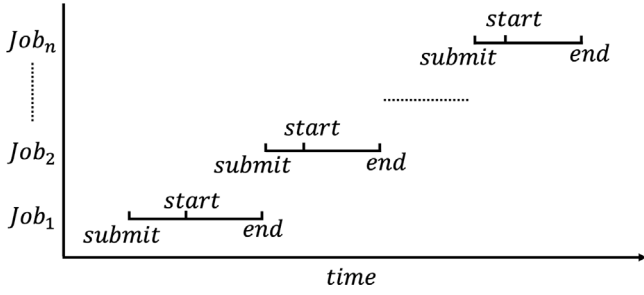


Fig. 7. Reactive job submission.

job scale with the maximum effective node hour consumption per unit time of TAT (see Section 6.2). All models are trained independently for each node scale (i.e., 4, 8, 16, 32 nodes) and evaluated on identical workloads.

Dual-objective optimization. This mode jointly optimizes turnaround time and node hour consumption. The agent receives a weighted sum as shown in Eq. (10) as the reward. During training, the scheduler is constrained by a fixed node hour budget and must determine an execution plan that maximizes the cumulative reward under this constraint. The three learning methods (i.e., DQN, PG, and AC) take machine states and job descriptions as input and output the distribution over candidate scales of 4, 8, 16, and 32 nodes.

In both modes, the reinforcement learning agent selects the compute scale of the job from {4, 8, 16, 32} nodes, dynamically adapting to changes in system load and workload characteristics. The performance of all policies is compared using metrics aligned with the optimization target (e.g., mean turnaround time and total node hour consumption). The results are discussed in Section 7.3.

6. Implementation

In this section, we discuss the details of implementing the SLM workload simulation, the greedy policy baseline, and the training configurations.

6.1. Emulating SLM workload

To emulate end-to-end SLM training workloads, we profile the performance of each workload on its documented scale and then project the overall node hour consumption to the smallest scale, e.g., four compute nodes. The Ciri agent defines a training workload with its node hour consumption and scaling profile. The simulation process starts by submitting the first job with the selected scale. Upon completion of the first job, the agent records the TAT, projects the raw NHC to the smallest scale, and then subtracts the actual consumption from the node hour balance. If the node hour balance is greater than zero, the agent will submit a second job, as shown in Fig. 7. This iterative process terminates when the node hour balance is zero.

Taking GPT-1.5B on Lonestar6 as an example, Fig. 1 shows its scaling efficiency, and the total node hour requirement is 16 nodes for 110 h. The 110-hour runtime was measured on 16 nodes [33] with the scaling efficiency of 50.7%, so we project the overall NHC to the minimum possible scale (i.e., 4 nodes). The required NHC budget is

$$\text{Budget}_{\text{GPT-1.5B}} = 16 \times 110 \times 0.507 = 892.32 \text{ node hours.}$$

At each step, the scheduler (agent) selects a job scale $a_i \in \{4, 8, 16, 32\}$. Each job runs for a maximum wall time of 48 h, consuming: $\text{NHC}_{\text{job}} = \text{scale} \times \text{duration} \times \text{eff}_{\text{scale}}$. The first job of GPT-1.5B runs on 16 nodes for 48 h and consumes: $16 \times 48 \times 0.507 = 389.4$ node hours. The remaining node hour is then $892.32 - 389.4 = 502.92$ node hours. Assuming the second job also runs on 16 nodes for 48 h, the balance is 113.52 node

hours. The agent iterates this process until the remaining node hour is zero.

Fig. 7 shows this reactive submission loop. As soon as one job finishes, the agent submits the next with updated system states and the remaining budget. The loop terminates when the node hour budget is exhausted. As can be seen, the number of jobs for a workload and the execution time of each job are dependent on the scale selection. This flexibility enables Ciri to trade off TAT for lower NHC.

6.2. The greedy policy

We implement a heuristic-based scheduler that selects a node scale by maximizing efficiency. For each scale $s \in \{4, 8, 16, 32\}$, the scheduler calculates a normalized benefit score defined as the ratio of node hour consumption to turnaround time. The node hour consumption is computed as

$$\text{NHC}_s = \text{ExecTime}_s \cdot s \cdot \text{Factor}_s$$

where Factor_s reflects the scaling efficiency at scale s , as shown in Fig. 1. The turnaround time is the sum of the predicted queue wait time and estimated execution time. The greedy policy chooses the scale that maximizes NHC per unit TAT:

$$\arg \max_s \frac{\text{NHC}_s}{\text{PredWait}_s + \text{ExecTime}_s}$$

The greedy policy delivers the lowest TAT, provided the predicted queue wait time. We incorporate the greedy policy in two distinct ways: (1) We use the greedy policy to isolate the queue wait time prediction capability of candidate base models (i.e., RF, XGB, and Trans) for the single-objective study in Section 7.2. (2) We combine the greedy policy with various base models to isolate the RL method contribution in queue wait time reduction in Section 7.2. The accuracy of the greedy policy is dependent on the accuracy of the base models, we have evaluated the mean queue wait time (QWT) predictor [10] and the quantile QWT predictor [12], which have significant higher prediction errors than the greedy policy.

6.3. Training configuration and cost

Ciri trains models in three places: (1) the queue wait time predictor, (2) the policy gradient method, and (3) the actor-critic method. We use the Adam optimizer with a fixed learning rate of 10^{-5} and the L1 (i.e., Mean Absolute Error) loss for all three training cases, where they are trained with 1000 episodes. Particularly for policy gradient, we use a baseline measured as the reward of the first step in every episode to enhance the convergence rate. The queue wait time predictors and the policy method are trained on the TACC Lonestar6 cluster using a single NVIDIA A100 GPU.

The Transformer-based predictor is trained separately on data collected at four, eight, sixteen, and thirty-two node scales. Each model takes approximately 8.2 h to converge, resulting in a total of 32.8 GPU-hours. Random Forest requires an average of 104.26 s per model, while XGBoost takes an average of 232.90 s per model. The training time cost of the base models with RL techniques is shown in Table 1.

7. Experiments and results

To validate the effectiveness and generality of Ciri, we design and conduct experiments using the job trace of the Lonestar6 system. To simplify the discussion, we assume each node has four GPUs. The job trace data is partitioned into two splits: the training set spans from 01/2023 to 06/2024, and the validation set covers 07/2024 to 12/2024. As shown in Section 4, the average queue time, monthly deviation between execution time and queue time limit, and the monthly average execution time exhibit similar patterns in training and validation ranges.

Table 1

Training cost (hrs) for Base models and RL methods.

Base	Model	DQN	PG	AC
RF	GPT-345M	5.4	4.9	4.6
	GPT-1.5B	10.2	9.3	8.7
	GPT-2.0B	12.7	11.5	10.7
XGB	GPT-345M	4.9	4.4	4.1
	GPT-1.5B	9.2	8.4	7.8
	GPT-2.0B	11.4	10.4	9.6
Trans	GPT-345M	10.8	9.8	9.3
	GPT-1.5B	20.5	18.6	17.7
	GPT-2.0B	25.3	23.0	21.9

We pretrain three GPT models using the OpenWebText dataset on 16 compute nodes (i.e., 64 NVIDIA GPUs) and measure the execution time: GPT-345M runs on 16 nodes for 22.6 h. GPT-1.5B runs on 16 nodes for 110.0 h. GPT-2.0B runs on 16 nodes for 156.0 h. The overall node hour consumption is projected to the four-node baseline using the measured scaling efficiency (see Fig. 1).

We first study the queue wait time prediction across scales of 4, 8, 16, and 32 nodes. SLM pretraining jobs usually run with a power-of-two number of GPUs, to guarantee a uniform distribution (determined by the local batch size and gradient accumulation) of workload across GPUs. Specifically, we evaluate ensemble methods, such as Random Forest (RF) and XGBoost (XGB), as well as the Transformer-based model. This initial stage lays the foundation by examining how accurately the methods can predict queue delays under dynamic machine loads. In the second study, we extend the queue wait time predictor by incorporating greedy and reinforcement learning strategies to enable single objective optimization for TAT. The experiments compare combinations of base models {RF, XGB, Transformer} and policies {Greedy, DQN, PG, AC} to minimize the overall turnaround time (TAT). The goal is to verify the correctness of Ciri's implementation and examine its capability in single-objective optimization. In the third study, we compare the candidate base model and RL method combinations for the dual-objective scenario. We sweep the parameter space of NHC weight across the values of {1.0, 0.9, 0.2, 0.1, 0}. These weights determine the relative importance of minimizing NHC against reducing TAT. We expect Ciri can efficiently trade TAT for NHC reduction across SLM workloads and machine busy levels.

The results show that, although the queue wait time predictors have modest errors, these errors do not propagate to decision policies for scale selection. For single-objective optimization, all candidate strategies can adjust scales to minimize TAT, while the actor-critic method with the Transformer base model achieves the lowest TAT in validation. For dual-objective optimization, all candidate methods can efficiently lower NHC with increased TAT. Again, combining the actor-critic method with the Transformer base model shows the most efficient tradeoff.

7.1. Queue wait time prediction

In this experiment, we focus on predicting the *queue wait time* of submitted jobs using three distinct models: Random Forest (RF), XGBoost (XGB), and Transformer (Trans). Each model is trained with varying numbers of nodes (4, 8, 16, and 32) to understand how scales impact predictive performance. Table 2 summarizes the results in terms of the predicted mean queue wait time (*Pred Mean*) and the mean absolute error (MAE). Across the scales, Random Forest has a percentage error (MAE/Mean) range of 33.3–45.5%. XGBoost has a range of 30.6–54.8%, while the Transformer-based model has 47.5–72.0%. None of the methods can accurately predict queue wait time. This is expected given the legacy works [10–12] and randomness in the job trace in Section 4.

Despite the high prediction errors of the base models, the goal of this research is to explore if RL methods can make correct actions

Table 2

Prediction Mean and MAE across different scales. Each model (Random Forest, XGBoost, and Transformer) is trained to predict job queue wait time on 4, 8, 16, and 32 nodes.

Model	Node count	Pred mean	MAE
RF	4	3.3	1.5
	8	4.0	1.8
	16	5.6	2.2
	32	11.1	3.7
XGB	4	3.1	1.7
	8	3.7	2.0
	16	5.4	2.3
	32	10.8	3.3
Trans	4	2.5	1.8
	8	3.0	1.9
	16	4.3	2.5
	32	9.9	4.7

Table 3

Comparison of Greedy, DQN (Deep Q-learning), AC (Actor-Critic), and PG (Policy Gradient) for GPT-2.0B Turnaround Time (TAT) Prediction. % low shows the TAT reduction compared to the greedy baseline with the respective base model.

Methods	Base	Val. (% low)	11/2024 (% low)
Greedy	RF	212.5	235.9
	XGB	196.5	220.4
	Trans	186.0	205.7
DQN	RF	195.8 (−7.8%)	212.7 (−9.8%)
	XGB	181.0 (−7.9%)	204.0 (−7.4%)
	Trans	172.3 (−7.4%)	189.3 (−8.0%)
PG	RF	194.9 (−8.3%)	206.8 (−12.3%)
	XGB	177.3 (−9.8%)	190.6 (−13.5%)
	Trans	167.4 (−10.0%)	176.3 (−14.3%)
AC	RF	186.3 (−12.3%)	198.4 (−15.9%)
	XGB	170.9 (−13.0%)	183.8 (−16.6%)
	Trans	166.3 (−10.6%)	172.6 (−16.1%)

with inaccurate predictions to lower TAT (see Section 7.2) and balance between TAT and NHC (see Section 7.3). Our hypothesis here is that the RL methods can make correct decisions as long as the QWT prediction errors are relatively correct, e.g., QWT is predicted longer for 32 nodes compared to four nodes. Sections 7.2 and 7.3 present the empirical validation of this hypothesis.

Next, we examine how these modest prediction errors translate into decision-making for scale selection.

7.2. Single goal optimization: Turnaround time

In the second experiment, we examine how different methods optimize the *turnaround time* (TAT) as a single objective. We compare four strategies: Greedy, DQN (Deep Q-learning), PG (Policy Gradient), and AC (Actor-Critic). We use the GPT-2.0B case and examine the TAT across the complete validation range and 11/2024, respectively.

Table 3 presents the average TAT (hrs) prediction for GPT-2.0B training with the candidate methods across the two validation sets. Within each method, the Transformer model consistently achieves a lower TAT than RF and XGB, although the queue wait time prediction has higher mean absolute errors (see Section 7.1). During the busy month of November 2024, methods using Policy Gradient and Actor-Critic are more effective in reducing TAT compared to the full validation time range. For example, Trans+AC achieves an average TAT of 172.6 h in 11/2024, which is 16.1% lower compared to the greedy baseline. In contrast, the TAT reduction for the full validation range is 10.6%. This observation suggests that Ciri is more effective in reducing TAT when the machine is heavily loaded. Across the base models of RF, XGB, and Transformer, Actor-critic achieves lower TAT monotonically in both validation tests compared with greedy, DQN,

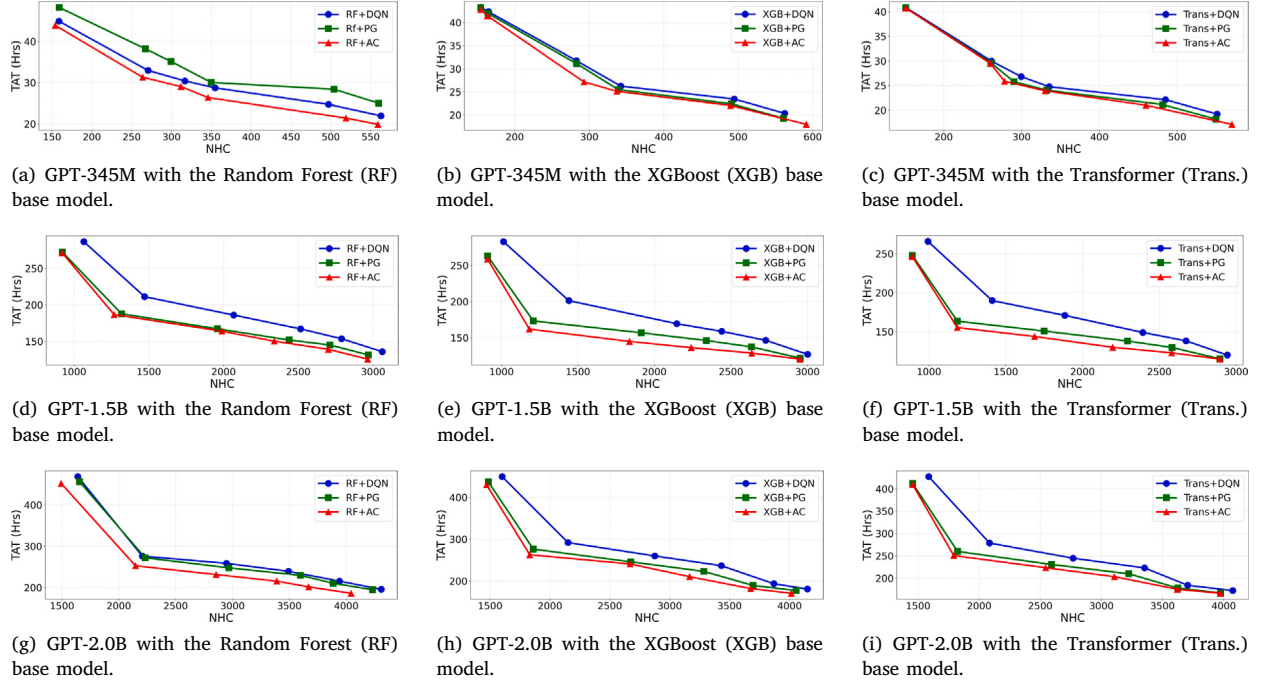


Fig. 8. Tradeoff between Turnaround Time (TAT) and Node Hour Consumption (NHC) on GPT-345M, GPT-1.5B, and GPT-2.0B. From left to right, the NHC weights are $\{1.0, 0.9, 0.5, 0.2, 0.1, 0.0\}$. The left and right endpoints of the DQN curves (blue) correspond to the measurements with fixed scale, i.e., 4 nodes and 32 nodes, respectively.

and PG. Among all twelve candidate combinations, Trans+AC achieves the lowest turnaround time for training GPT-2.0B. Despite the modest errors in queue wait time predictions from the base models, Ciri can still make accurate scale decisions to minimize TAT by considering both queue wait time and execution time.

7.3. Dual-objective optimization: TAT vs. NHC

In the third experiment, we evaluate the capability and efficiency of Ciri in navigating the tradeoff space between *turnaround time* (TAT) and *node hour consumption* (NHC) across three GPT models: GPT 345M, GPT 1.5B and GPT 2.0B. We simulate SLM workload execution with nine candidate combinations of {RF, XGB, Transformer} and {DQN, PG, AC} and vary the *NHC weight* in the range of $\{1.0, 0.9, 0.5, 0.2, 0.1, 0.0\}$ to balance the two objectives. Fig. 8 shows the tradeoff curve comparisons of DQN, PG, and AC for GPT-345M, GPT-1.5B, and GPT-2.0B. From left to right on each curve, the NHC weights are $\{1.0, 0.9, 0.5, 0.2, 0.1, 0.0\}$. Each figure illustrates the non-linear tradeoff space of NHC and TAT controlled by the NHC weight using a candidate method. The curves that approach the lower left in Fig. 8 are more efficient in the tradeoff between NHC and TAT. It is worth noting that the two endpoints of the curves (blue) with the DQN method are the baselines with fixed scale. The left endpoint is associated with an NHC weight of 1.0, while the right endpoint correlates to an NHC weight of 0.0. The two endpoints are measured using fixed scales of 4 and 32 nodes, respectively. Across all combinations, Trans+AC achieves the most efficient tradeoff as shown in Fig. 8.

7.3.1. Comparison of base models

Fig. 9 shows the normalized tradeoff efficiency among the combination of three base models (Random Forest, XGBoost, and Transformer) and learning methods (DQN, PG, and AC) across three workloads: GPT-345M, GPT-1.5B, and GPT-2.0B. The vertical axis is the absolute ratio between the changes in NHC (Δnhc) and TAT (Δtat). The ratio with a value of one indicates that the learning method can lower the NHC by 1% by increasing the TAT by 1%. Higher ratios reflect that the learning

methods are more efficient in trading TAT for NHC reduction. The ratio is measured by varying the NHC weight from 0.0 to 0.1.

For GPT-345M, when fixing the learning method to DQN, PG, or AC, the RF model achieves the highest absolute ratio, indicating the most efficient tradeoff. For GPT-1.5B, the Transformer base model with AC shows the highest ratio of 1.6. For GPT-2.0B, the Transformer base model with AC achieves the highest ratio of 1.7 across all three learning methods, indicating the most efficient tradeoff for larger models. The Transformer base model is more effective in large model training, as it requires larger node hour consumption. The GPT-345M training requires only 16 nodes for 22.6 h, which can be completed with a single job. In this case, GPT-345M cannot fully leverage Ciri's capability in continuous scale selection to balance between TAT and NHC.

7.3.2. Comparison of learning methods

Fig. 10 shows the absolute ratio between the changes in NHC and TAT with the NHC weight changing from 0.0 to 0.1. Among the nine candidate methods, AC consistently achieves a higher ratio compared with DQN and PG, except using XGB as the base model for GPT-345M and using RF as the base model for GPT-1.5B. By changing NHC weight from 0.0 to 0.1, combining the Transformer and AC can reduce the NHC by 19.4% with a 22.7% higher TAT for GPT-345M. For GPT-1.5B, the same combination can reduce NHC by 10.9% with a 6.9% increase in TAT. For GPT-2.0B, the tradeoff is 8.9% NHC reduction and 5.4% TAT increase.

By changing the NHC weight from 0.0 to 0.2, Trans+AC can reduce the NHC by 21.8% with a 22.3% TAT increase for GPT-2.0B. For GPT-1.5B, Trans+AC trades off 13.1% TAT increase for 21.8% lower NHC.

AC achieves the most efficient tradeoff between NHC and TAT among all learning methods, particularly when combined with the Transformer base model. The policy can be tuned with a single hyperparameter (NHC weight) to adapt to different operational priorities. Thus, Ciri uses the Transformer base model and actor-critic method as defaults.

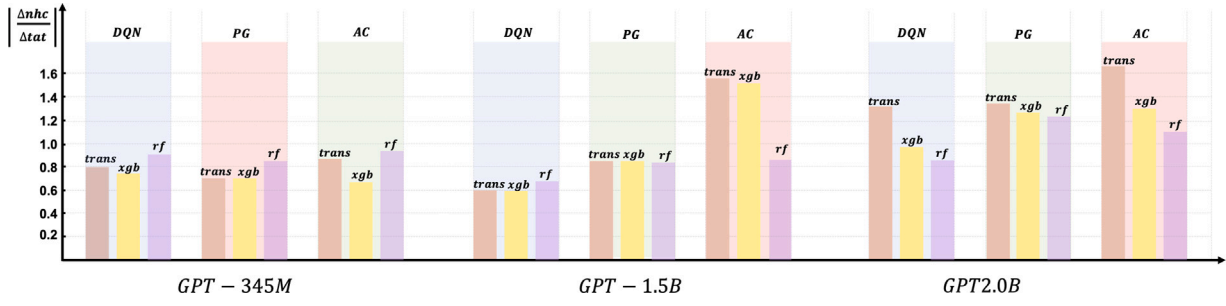


Fig. 9. Comparison between base models of Trans, XGB, and RF with Fixed RL Methods of DQN, PG, and AC. The vertical axis represents the normalized tradeoff efficiency, which is the absolute percentage ratio of Node Hour Consumption (NHC) increase to Turnaround Time (TAT) reduction as the NHC weight varies from 0.0 to 0.1.

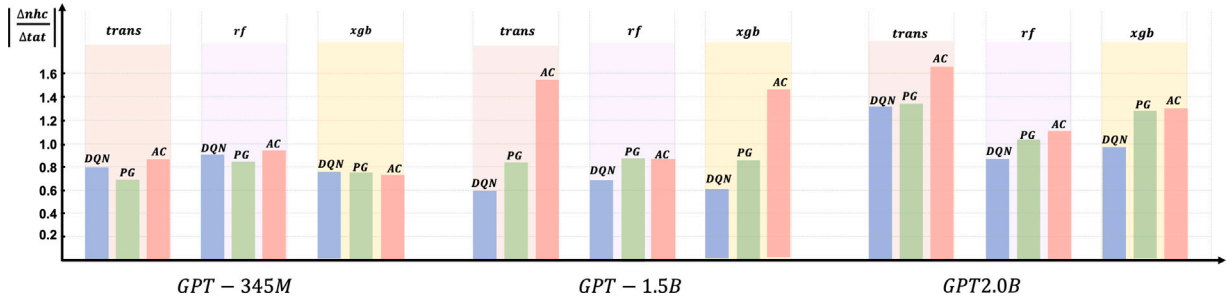


Fig. 10. Comparison between learning methods of DQN, PG, and AC with Fixed base models of Trans, XGB, and RF. The vertical axis represents the normalized tradeoff efficiency, which is the absolute percentage ratio of Node Hour Consumption (NHC) increase to Turnaround Time (TAT) reduction as the NHC weight varies from 0.0 to 0.1.

7.3.3. Evaluation on Frontera

Fig. 11 shows the absolute ratio between the changes in Node Hour Consumption (NHC) and Turnaround Time (TAT) as the NHC weight changes from 0.0 to 0.1 on the Frontera cluster. For GPT-345M, the AC with the Transformer base model achieves the most efficient tradeoff: it reduces NHC by 18.1% at the cost of a 22.5% increase in TAT. This ratio is higher than with XGBoost (−14.1% NHC, 22.2% TAT) and RF (7.2% NHC, 8.2% TAT), demonstrating AC's advantage even on small workloads. For GPT-1.5B, the Transformer+AC reduces NHC by 11.6% with a 7.0% increase in TAT, outperforming XGBoost (−11.5%, 7.1%) and RF (−8.5%, 10.5%). For GPT-2.0B, Transformer+AC reduces NHC by 7.8% while increasing TAT by 5.3%. This is a more efficient tradeoff than XGBoost (−9.6% NHC, 12.0% TAT) and RF (−9.2% NHC, 8.2% TAT). Overall, Transformer+AC provides the most efficient and stable tradeoffs across all job sizes.

These results show the effectiveness of Ciri in balancing dual-objective scheduling optimization, especially on over-committed GPU clusters. So we use Transformer+AC as the default scheduling policy.

7.4. Robustness to changing environments

Ciri effectiveness depends on the accuracy of the QWT predictor. In this section, we aim to quantify this impact by comparing Ciri's performance across the full validation range (07/2020-12/2024) with that in the busiest month, 11/2024. Table 3 shows higher GPT-2.0B QWT prediction errors for AC with the Transformer base model, i.e., −16.1% vs. −10.6%. Across the validation range, with NHC weights of from 0.0 to 0.1, the ratio of NHC reduction to TAT increase is 1.65, compared to 0.88 in 11/2024. For GPT-1.5B, the NHC and TAT ratio is 1.57 for the complete validation range, compared to 1.49 in 11/2024. These results indicate that Ciri's effectiveness depends on the QWT prediction accuracy.

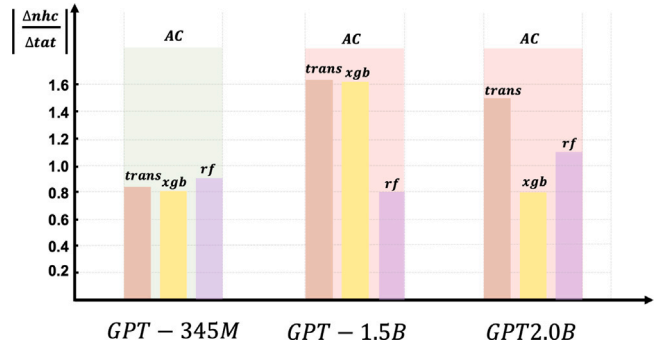


Fig. 11. Comparison between Base Models of Trans, XGB, and RF with AC Method on Frontera. The vertical axis represents the normalized tradeoff efficiency, which is the absolute percentage ratio of Node Hour Consumption (NHC) increase to Turnaround Time (TAT) reduction as the NHC weight varies from 0.0 to 0.1.

8. Discussion

Ciri is motivated by SLM training workloads, and we evaluate the effectiveness of Ciri using the Lonestar6 job trace from 01/2023 to 12/2024. We discuss how Ciri can be applied to other machines and HPC workloads amid the hardware evolution of GPU clusters.

8.1. Generality of Ciri

Ciri provides a general solution that enables users to trade off turnaround time for reduced node hour consumption efficiently. Ciri is designed to generalize across different workloads and system conditions. We examine the generalization of Ciri by strictly following the training/validation data partitioning principle. We evaluate the

tradeoff effectiveness of Ciri using three GPT models with representative scaling curves and verify the efficacy. Across GPT-345M, GPT-1.5B, and GPT-2.0B, Ciri achieves effective tradeoffs between TAT and NHC under the same training procedures and hyperparameters. The experiments confirm the effectiveness of Ciri on Lonestar6 and Frontera.

Ciri can be easily extended to LLM pretraining. Modern LLM pretraining leverages 3D or 4D parallel strategy, which overlays data-parallel over pipeline-parallel, tensor-parallel, and context-parallel. The GPT models studied in this paper are sufficiently small so that scaling profiles rely solely on data parallelism. Ciri can be extended to support LLM, given the 3D/4D scaling profiles along the data-parallel dimension, with a baseline measured using the minimum number of GPUs to host a single model.

8.2. Applicability of Ciri

The GPU compute capability increases at a faster rate than interconnect bandwidth. The BF16 performance has improved from 624 TFLOPS to 1979 TFLOPS and to 3.5 PFLOPS across the NVIDIA A100, H100, and B100 GPUs. During the same time period, the interconnect bandwidth, such as Infiniband, increases from 200 Gbps to 800 Gbps per link. The increased disparity makes communication a more significant performance bottleneck for modern SLM and LLM workloads. This can be evidenced by the 41% BF16 MFU (Model FLOPS Utilization) of GPT 1.7B model training [34]. In such a situation, Ciri will provide an effective solution for those deep learning practitioners with tight node-hour budgets on national computing infrastructures. Even though we use SLM training workloads to examine the effectiveness of Ciri, the efficient TAT and NHC tradeoff is applicable to any numerical simulation or data analytic workload that can run at multiple scales. Computing centers can provide Ciri as a solution for users to reduce node hour consumption with a quantified tradeoff in turnaround times. The overall machine utilization will also be enhanced if the majority of workloads are running at more efficient scales.

8.3. Deployment overhead

The deployment overhead for Ciri includes model training and data preprocessing. The base models across scales are a two-layer transformer neural network, and the AC policy head is a two-layer MLP. The model can be hosted on a low-end GPU, e.g., an NVIDIA GTX 1080 Ti with 12 GB of RAM. Ciri requires a data stream that collects SLURM queue states every 10 min. For Lonestar6 or Frontera, Ciri can be deployed on a standalone computer with a low-end GPU and a data stream with authenticated access to the supercomputer's login node. The total node hour cost for Ciri consists of 11.0 CPU hours (AMD EPYC 7763) for offline data generations, 32.8 GPU hours (A100) for queue wait time predictors, and 21.9 GPU hours (A100) for AC. For other GPU clusters, users need to train the base models and the AC policy with job traces, which may need 100 s of GPU hours.

8.4. Implications to overall machine utilization

Ciri is a user tool that enables an efficient tradeoff between TAT and NHC. If adopted by a few users, Ciri can reduce the overall NHC of the experiments and the resource utilization is higher with Ciri, as it explores opportunities to run at smaller scales without significantly increasing the TAT. If adopted by the majority of users of a GPU cluster, these users will run at a higher machine utilization for the same aforementioned reason, as will the machine. Given the excessively long queue observed in Section 4, more jobs can run at the same time on the machine, which enhances the responsiveness of the GPU cluster. The overall machine will operate at 100% utilization on the node level.

9. Conclusion

We present Ciri, a reinforcement learning-enabled user-level job scheduler, which can efficiently trade turnaround times for lower node hour consumption for SLM training in GPU clusters. Ciri leverages the insights from the SLM workload scaling curve, and the agent is trained using historical job traces. Experiments on three workloads, GPT-345M, GPT-1.5B, and GPT2.0B, verify Ciri's correctness and effectiveness. Specifically for GPT-1.5B and GPT-2.0B, Ciri can reduce the overall node hour consumption by 0.9–1.8% percent with every 1% increase in turnaround time. Among all learning methods, the Actor–Critic (AC) method offers the most efficient trade-off, especially when combined with a Transformer-based wait time predictor. From a practical point of view, Ciri can help users reduce monetary costs on HPC and Cloud platforms by improving resource utilization. For example, reducing NHC by 20% can directly translate into a 20% decrease in GPU rental fees. Given the existing scalability of SLM and LLM workloads and the trends in GPU and interconnect evolution, Ciri will provide an effective solution for deep learning practitioners with tight node-hour budgets on national computing infrastructures.

CRediT authorship contribution statement

Jingxin Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation. **Shuyuan Fan:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation. **Zhao Zhang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the NSF ICICLE AI Institute (OAC-2112606). The authors acknowledge the Texas Advanced Computing Center (TACC) at The University of Texas at Austin for providing computational resources that have contributed to the research results reported within this paper.

Data availability

Data will be made available on request.

References

- [1] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, *Adv. Neural Inf. Process. Syst.* 35 (2022) 27730–27744.
- [2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q.V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, *Adv. Neural Inf. Process. Syst.* 35 (2022) 24824–24837.
- [3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P.D.O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, 2021, arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- [4] G. Ahdrizt, N. Bouatta, C. Floristean, S. Kadyan, Q. Xia, W. Gerecke, T.J. O'Donnell, D. Berenberg, I. Fisk, N. Zanichelli, et al., OpenFold: Retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization, *Nature Methods* 21 (8) (2024) 1514–1524.

- [5] N. Ho, C.N. Ellington, J. Hou, S. Addagudi, S. Mo, T. Tao, D. Li, Y. Zhuang, H. Wang, X. Cheng, L. Song, E.P. Xing, Scaling dense representations for single cell with transcriptome-scale context, *bioRxiv* (2024).
- [6] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y.C. Lin, P. Molchanov, Small language models are the future of agentic AI, 2025, *arXiv preprint arXiv:2506.02153*.
- [7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems, NeurIPS, Vol. 33*, Curran Associates, Inc., Red Hook, NY, USA, 2020, pp. 1877–1901, URL: <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc64967418bfb8ac142f64a-Paper.pdf>.
- [8] ACCESS Program, Advanced cyberinfrastructure coordination ecosystem: Services & support, 2025, URL: <https://access-ci.org/>. (Accessed 05 October 2025).
- [9] National Artificial Intelligence Research Resource Task Force, Strengthening and democratizing the U.S. artificial intelligence innovation ecosystem, 2023, URL: <https://www.ai.gov/wp-content/uploads/2023/01/NAIRR-TF-Final-Report-2023.pdf>.
- [10] W. Smith, V. Taylor, I. Foster, Using run-time predictions to estimate queue wait times and improve scheduler performance, in: *Job Scheduling Strategies for Parallel Processing: IPPS/SPDP'99Workshop, JSSPP'99 San Juan, Puerto Rico, April 16, 1999 Proceedings 5*, Springer, 1999, pp. 202–219.
- [11] D. Nurmi, A. Mandal, J. Brevik, C. Koelbel, R. Wolski, K. Kennedy, Evaluation of a workflow scheduler using integrated performance modelling and batch queue wait time prediction, in: *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*, 2006, pp. 119–es.
- [12] D.C. Nurmi, J. Brevik, R. Wolski, QBETS: Queue bounds estimation from time series, *ACM SIGMETRICS Perform. Eval. Rev.* 35 (1) (2007) 76–101.
- [13] O. Sonmez, N. Yigitbasi, A. Iosup, D. Epema, Trace-based evaluation of job runtime and queue wait time predictions in grids, in: *Proceedings of the 18th ACM International Symposium on High Performance Distributed Computing*, 2009, pp. 111–120.
- [14] Y. Fan, Z. Lan, T. Childers, P. Rich, W. Allcock, M.E. Papka, Deep reinforcement agent for scheduling in HPC, in: *2021 IEEE International Parallel and Distributed Processing Symposium, IPDPS, IEEE, 2021*, pp. 807–816.
- [15] Y. He, G. Wu, Y. Chen, W. Pedrycz, A two-stage framework and reinforcement learning-based optimization algorithms for complex scheduling problems, 2021, *arXiv preprint arXiv:2103.05847*.
- [16] J. Ryu, J. Eo, Network contention-aware cluster scheduling with reinforcement learning, in: *2023 IEEE 29th International Conference on Parallel and Distributed Systems, ICPADS, IEEE, 2023*, pp. 2742–2745.
- [17] G. Infantes, S. Roussel, P. Pereira, A. Jacquet, E. Benazera, Learning to solve job shop scheduling under uncertainty, in: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, Springer, 2024*, pp. 329–345.
- [18] Q. Ding, P. Zheng, S. Kudari, S. Venkataraman, Z. Zhang, Mirage: Towards low-interruption services on batch GPU clusters with reinforcement learning, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '23, Association for Computing Machinery, New York, NY, USA, ISBN: 9798400701092, 2023*, <http://dx.doi.org/10.1145/3581784.3607042>.
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems, NeurIPS, Curran Associates Inc., Red Hook, NY, USA, ISBN: 9781510860964, 2017*, pp. 6000–6010.
- [20] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Židek, A. Potapenko, et al., Highly accurate protein structure prediction with AlphaFold, *Nature* 596 (7873) (2021) 583–589.
- [21] M. Zvyagin, A. Brace, K. Hippe, Y. Deng, B. Zhang, C.O. Bohorquez, A. Clyde, B. Kale, D. Perez-Rivera, H. Ma, et al., GenSLMs: Genome-scale language models reveal SARS-CoV-2 evolutionary dynamics, *Int. J. High Perform. Comput. Appl.* 37 (6) (2023) 683–705.
- [22] M. Andrychowicz, L. Espeholt, D. Li, S. Merchant, A. Merose, F. Zyda, S. Agrawal, N. Kalchbrenner, Deep learning for day forecasts from sparse observations, 2023, *arXiv preprint arXiv:2306.06079*.
- [23] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [24] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484–489.
- [25] M. Papini, D. Binaghi, G. Canonaco, M. Pirotta, M. Restelli, Stochastic variance-reduced policy gradient, in: *International Conference on Machine Learning, PMLR, 2018*, pp. 4026–4035.
- [26] T. Pierrot, V. Macé, F. Chalumeau, A. Flajolet, G. Cideron, K. Beguir, A. Cully, O. Sigaud, N. Perrin-Gilbert, Diversity policy gradient for sample efficient quality-diversity optimization, in: *Proceedings of the Genetic and Evolutionary Computation Conference, 2022*, pp. 1075–1083.
- [27] V.R. Konda, J.N. Tsitsiklis, Actor-critic algorithms, in: *Advances in Neural Information Processing Systems, Vol. 12*, 2000, pp. 1008–1014.
- [28] SchedMD, SLURM Multi-factor priority plugin, 2025, URL: https://slurm.schedmd.com/priority_multifactor.html. (Accessed 08 April 2025).
- [29] Z. Mo, H. Xu, C. Xu, Heet: Accelerating elastic training in heterogeneous deep learning clusters, in: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Vol. 2, 2024*, pp. 499–513.
- [30] Z. Mo, H. Xu, W.C. Lau, Optimal resource efficiency with fairness in heterogeneous GPU clusters, in: *Proceedings of the 25th International Middleware Conference, 2024*, pp. 36–48.
- [31] Y. Guan, Y. Liu, Y. Li, X. Xu, HierRL: Hierarchical reinforcement learning for task scheduling in distributed systems, in: *2022 International Joint Conference on Neural Networks, IJCNN, IEEE, 2022*, pp. 1–8.
- [32] Y. Guan, M. Xie, Y. Li, X. Xu, Attentive reinforcement learning for scheduling problem with node auto-scaling, in: *2022 IEEE International Conference on Systems, Man, and Cybernetics, SMC, IEEE, 2022*, pp. 2533–2539.
- [33] M. Zheng, Z. Zhang, Radius: Range-based gradient sparsity for large foundation model pre-training, in: *Eighth Conference on Machine Learning and Systems, 2025*.
- [34] NVIDIA, Megatron-LM github, 2018, <https://github.com/NVIDIA/Megatron-LM>.